

Shell Scripting in Linux: Beginner to Advanced

Table of Contents

1. Introduction to Shell Scripting
2. Setting Up Your Environment
3. Basic Shell Commands
4. Working with Variables
5. Input and Output
6. Conditional Statements
7. Looping Constructs
8. Functions in Shell
9. Command-Line Arguments
10. File Testing and Permissions
11. String and Arithmetic Operations
12. Arrays in Shell
13. Error Handling and Debugging
14. Regular Expressions and Text Processing
15. Input/Output Redirection
16. Pipes and Filters
17. Cron Jobs and Automation
18. Process Management
19. Creating Interactive Scripts
20. Advanced Bash Features
21. Real-World Projects
22. Best Practices and Optimization

Shell Scripting in Linux: Beginner to Advanced

1. Introduction to Shell Scripting

Shell scripting is writing a series of command-line instructions saved in a file that can be executed by the shell. It automates tasks and simplifies repetitive processes.

What is a Shell?

A shell is a command-line interpreter that provides a user interface for the Unix/Linux operating system.

Types of Shells:

- Bourne Shell (sh)
- Bourne Again Shell (bash)
- C Shell (csh)
- Korn Shell (ksh)

2. Setting Up Your Environment

Tools Needed:

- Terminal (Linux or macOS)
- Bash Shell
- Text Editor (Nano, Vim, VS Code)

Creating Your First Script:

```
#!/bin/bash
```

```
echo "Hello, World!"
```

Save it as hello.sh and run:

Shell Scripting in Linux: Beginner to Advanced

```
chmod +x hello.sh
```

```
./hello.sh
```

3. Basic Shell Commands

Shell commands are the building blocks of any shell script.

Commonly used commands:

- ls : List directory contents
- pwd : Print working directory
- cd : Change directory
- mkdir : Make directories
- rmdir : Remove empty directories
- rm : Remove files or directories
- cp : Copy files or directories
- mv : Move or rename files or directories
- echo : Display a line of text
- man : Display manual of commands

4. Working with Variables

Variables in shell scripts store data for later use.

Syntax:

```
variable_name=value
```

Shell Scripting in Linux: Beginner to Advanced

Example:

```
name="John"
```

```
echo $name
```

Environment variables:

- PATH

- HOME

- USER

Use ``export`` to set environment variables:

```
export MYVAR=value
```

5. Input and Output

Reading user input:

```
read -p "Enter your name: " name
```

```
echo "Hello, $name"
```

Redirecting output:

- `>` : Redirect stdout to a file

- `>>` : Append stdout to a file

- `<` : Redirect input from a file

Example:

Shell Scripting in Linux: Beginner to Advanced

```
echo "Sample text" > output.txt
```

```
cat < output.txt
```

6. Conditional Statements

Conditional statements allow decisions in scripts.

if-then statement:

```
if [ condition ]; then
```

```
    command
```

```
fi
```

if-then-else:

```
if [ condition ]; then
```

```
    command1
```

```
else
```

```
    command2
```

```
fi
```

Comparison operators:

- -eq : equal to

- -ne : not equal to

- -gt : greater than

- -lt : less than

- -ge : greater than or equal to

Shell Scripting in Linux: Beginner to Advanced

- -le : less than or equal to

7. Looping Constructs

Loops help repeat commands.

for loop:

```
for i in 1 2 3; do
```

```
    echo $i
```

```
done
```

while loop:

```
i=1
```

```
while [ $i -le 3 ]; do
```

```
    echo $i
```

```
    i=$((i+1))
```

```
done
```

until loop:

```
i=1
```

```
until [ $i -gt 3 ]; do
```

```
    echo $i
```

```
    i=$((i+1))
```

```
done
```